



COS 132 – Imperative Programming

Foundation Test #3

ST2 Programme

Module:	COS 132 – Imperative Programming
Assessment:	Foundations Test #3 (ST2 Programme)
Set by:	Swift Tuition
Duration:	30 minutes
Total Marks:	25
Difficulty:	Hard – exam-realistic ST1 simulation

Programme note. This material is for Day 1 of the 7-day Swift Tuition COS 132 ST2 programme. The premium course is available at www.swifttuition.com/tuks/cos132. Learners watching before ST2 2026 may use coupon code **200COS132** at checkout for R200 off the full price. All three foundation tests are included; one test can be taken now and the remaining tests anytime after ST2. Where units are mentioned, they refer to Swift Tuition's learning-unit split of the module's lecture content, covering lectures 1–16.

1 Instructions

1. Read each question carefully and answer all of them.
2. Write your answers in a separate file or on paper. Do not edit this question paper.
3. The assessment comprises **4** questions over **25** marks.
4. This is a closed-book assessment. No internet, no notes, no compiler.
5. You have **30 minutes** to complete the paper.

Question:	1	2	3	4	Total
Marks:	8	6	5	6	25

Name and surname: _____

Student number: _____

Question 1

8 marks

(a) For each of the statements below, state whether the statement is True or False.

- i. The expression `static_cast<char>(65)` evaluates to the character 'A'. (1)
- ii. In C++, an `int` and a `char` cannot be compared directly using `==` because they have different types. (1)
- iii. A function declared as `void f(const int x)` is allowed to read `x` but not modify it inside the function body. (1)
- iv. The C++ statement `int x = 5, y = x + 2;` declares two variables and initialises them in a single statement. (1)

(b) For each of the questions below, choose the most correct option.

- i. What is the value of `result` after the following code runs? (1)

```
int a = 5;
int b = 2;
double result = a / b + 0.5;
```

- A. 2.5
- B. 3.0
- C. 3.5
- D. 0.5

- ii. Consider the following code: (1)

```
int i = 1;
while (i < 5) {
    i = i + 1;
}
cout << i;
```

- What is printed? A. 4
- B. 5
 - C. 6
 - D. The loop runs forever

- iii. Which of the following correctly explains why a function defined like this might cause a problem? (1)

```
double divide(int a, int b) {
    return a / b;
}
```

- A. Functions can't return `double` if both parameters are `int`.
 - B. Integer division is performed before the result is converted to `double`, so fractional precision is lost.
 - C. The function will fail to compile.
 - D. The result is always rounded up.
- iv. In a multi-file C++ project, what is the role of a `#ifndef ... #define ... #endif` block in a header file? (1)
- A. It tells the linker which file to use.
 - B. It prevents the same header from being included more than once in a single translation unit.
 - C. It speeds up compilation by skipping the file entirely.
 - D. It makes the header file private to one `.cpp` file.

Question 2

6 marks

For each of the following programs, study the code and provide the exact output the program would produce.

(a)

```
#include <iostream>
using namespace std;

int compute(int a, int b = 3) {
    return a + b;
}

double compute(double a, double b) {
    return a * b;
}

int main() {
    cout << compute(2) << " ";
    cout << compute(2, 4) << " ";
    cout << compute(2.0, 4.0);
    return 0;
}
```

(3)

(b)

```
#include <iostream>
using namespace std;

int main() {
    int n = 4;
    int total = 0;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++) {
            total = total + j;
        }
    }
    cout << total;
    return 0;
}
```

(3)

Question 3

5 marks

Consider a multi-file C++ project organised as follows:

****Geometry.h****

```
#ifndef Geometry_h
#define Geometry_h

double areaOfRectangle(double width, double height);
double areaOfCircle(double radius);

#endif
```

****Geometry.cpp****

```
#include "Geometry.h"

double areaOfRectangle(double width, double height) {
    return width * height;
}

double areaOfCircle(double radius) {
    // implementation needed
}
```

****main.cpp****

```
#include <iostream>
#include "Geometry.h"
using namespace std;

int main() {
    cout << areaOfRectangle(3.0, 4.0) << endl;
    cout << areaOfCircle(5.0) << endl;
    return 0;
}
```

A constant PI is defined in a separate file Constants.h (assume `const double PI = 3.14159;` is declared there).

(a) Provide the implementation of `areaOfCircle` in `Geometry.cpp`. The formula is πr^2 . Include any additional `#include` directive that becomes necessary. (2)

(b) Write the makefile entries needed to:

- i. Compile `Geometry.o` from `Geometry.cpp` (and its dependencies). Indicate where any **mandatory tab(s)** appear in the entry. (2)

- ii. Link the program into an executable called `Main`, given that `main.o` and `Geometry.o` already exist. (1)

Question 4

6 marks

Consider the following Logo procedure:

```
TO PRINT_GRID :ROWS :COLS
  REPEAT :ROWS [
    REPEAT :COLS [
      PRINT1 "*"
    ]
    PRINT ""
  ]
END
```

The procedure prints a grid of :ROWS by :COLS asterisks. PRINT1 prints without a newline; PRINT "" prints an empty line (a newline). For example, PRINT_GRID 3 4 would print:

```
****
****
****
```

(a) Convert the Logo procedure into an equivalent C++ function called printGrid that takes two int parameters and produces the same output. You do not need to write a main function. Assume iostream and the std namespace are available. (4)

(b) Now suppose you wanted to only print rows whose row index (starting at 1) is odd. Modify your C++ function so that, for example, `printGrid(5, 3)` would print rows 1, 3, and 5 only:

```
***  
***  
***
```

Provide only the modified C++ function. (2)

End of paper · 25 marks · 30 minutes